LeakBench-Tab: Separating Contamination Validity, Statistical Detectability, and Model Exploitability in Tabular Learning

Anonymous submission

INTERNAL DRAFT — RESULTS BLOCKED. The complete confirmatory `paper_claims.json` has not passed the fail-closed paper-number generator. No empirical value in this PDF is submission-ready.

Abstract

Prediction-time feature contamination can invalidate tabular model evaluation, yet an invalid feature need not be statistically conspicuous or useful to a particular learner. We present LeakBench-Tab, a measurement benchmark that separates prediction-time validity (C), statistical detectability (D), and model exploitability (X). Its frozen controlled design crosses 20 panel tasks, 11 mechanisms, 5 strengths, 5 model families, and 5 injection seeds, for 27,500 paired model-training cells. Mechanism-level construction invariants enforce the declared prediction boundary, and hierarchical inference treats the task rather than the cell as the independent unit. A separate 22,000-cell sensitivity matrix compares 4 oracle-blind, pre-test development rankers; mutual information is the frozen primary diagnostic. Empirical findings are intentionally withheld until the complete confirmatory release passes the frozen integrity and claim-status checks.

1 Introduction

Prediction-time leakage arises when a model is trained or evaluated with information that would not be valid at its declared decision boundary. Direct copies of the target are obvious, but practical contamination also enters through post-outcome fields, future-window aggregates, and entity or source statistics whose legitimacy depends on lineage and timing (Kaufman, Rosset, and Perlich 2011; Kapoor and Narayanan 2023; Davis et al. 2024). Such fields can make offline performance optimistic and comparisons between models misleading.

Association alone does not establish invalidity. Conversely, an invalid field may be hard to locate from samples alone, and a near-zero performance change does not make that field legitimate. We therefore separate three questions: *contamination validity* (C), determined by the prediction boundary; *statistical detectability* (D), measured as localization quality; and *model exploitability* (X), measured by paired permissive-versus-strict performance inflation.

We ask: **RQ1:** Which prediction-time contamination mechanisms can oracle-blind, pre-test development diagnostics locate, and how diagnostic-conditional is that conclusion? **RQ2:** Which mechanisms are exploited across linear, tree-ensemble, boosting, and neural tabular learners? **RQ3:** Which C/D/X profiles persist across mechanism strengths, model families, and explicitly bounded real-data case studies?

Our contribution is a benchmark-level operationalization rather than a new definition of leakage or a new detector. First, LeakBench-Tab records C, D, and X as distinct estimands tied to a declared prediction boundary. Second, it provides 11 construction-audited mechanisms on 20 deterministic controlled panel tasks, with paired strict and permissive evaluation. Third, it freezes exact task-level inference for the directional category contrast and task-hierarchical uncertainty analyses for descriptive mechanism, diagnostic, and model-family summaries. Fourth, it supplements controlled evidence with 5 fixed real-data case studies whose scope is explicitly limited to bounded consistency checks. Metadata localization and governance interventions are outside the confirmatory claim set reported here.

2 Related Work

Leakage semantics and scientific validity. Prediction-time legitimacy and the "no time machine" principle precede this work (Kaufman, Rosset, and Perlich 2011). Later analyses connect leakage to failures of scientific reproducibility and domain-specific label leakage (Kapoor and Narayanan 2023; Davis et al. 2024). Controlled neuroimaging audits also show that different leakage forms can produce large or minor performance inflation (Rosenblatt et al. 2024). Most closely, a recent tabular study applies 29 workflow perturbations across 2,047 i.i.d. and 129 temporal datasets to compare leakage-type effect sizes (Roth 2026). That work provides substantially broader dataset coverage; our complementary resolution is within-task feature localization, construction-audited prediction boundaries, and paired diagnostic/model profiles for eleven fixed mechanisms. LeakBench-Tab does not redefine these concepts or claim that variable inflation is new. Its distinct scope is a domain-agnostic tabular mechanism registry that jointly evaluates semantic validity, feature-level localization, and paired inflation across diagnostic and downstream model families.

Tabular model evaluation. Broad comparisons such as TableShift and TabZilla study distribution shift and model-family performance across tabular tasks (Gardner, Popovic, and Schmidt 2023; McElfresh et al. 2023). Other work establishes strong tree baselines and modern neural alternatives (Grinsztajn, Oyallon, and Varoquaux 2022; Gorishniy, Kotelnikov, and Babenko 2025). Our axis of comparison is different: every learner receives paired views of the same frozen injected task, so the estimand is contamination harm rather than unpaired predictive rank.

Diagnostics versus exploitation. The distinction between contamination, memorization, and downstream exploitation is not new: Magar and Schwartz (2022) showed in NLP pretraining that memorized contaminated examples need not be exploited after fine-tuning. Our use of "exploitability" therefore claims no conceptual priority. We operationalize a different setting: prediction-time tabular feature contamination C, localization of encoded contaminated columns D, and paired strict-versus-permissive degradation X. Static program analysis offers a complementary route by detecting leakage-prone data-science code before execution (Subotić, Bojanić, and Stojić 2022); our D diagnostics instead localize contaminated columns in a frozen task. Feature-level precision–recall summaries are appropriate when contaminated fields are sparse (Saito and Rehmsmeier 2015), but successful localization does not imply that every learner will exploit the field. We use mutual information as the primary D measure and compare it with absolute correlation, logistic coefficient magnitude, and random-forest permutation importance. All four are oracle-blind and pre-test: the first three use training rows only, while the forest is fitted on training rows and permutation importance is evaluated with the frozen validation labels. Their disagreement is descriptive sensitivity, not a procedure for selecting the best ranker after seeing contamination labels. X is reported separately for each downstream model. The benchmark is measurement infrastructure, not a proposal for a universally superior leakage detector or governance policy.

3 Problem Setup and the C/D/X Framework

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ be a binary tabular task with declared prediction time t_p. A field may be invalid because it becomes available after t_p, encodes future or outcome information, or violates a declared lineage or population boundary. The *strict* view removes all mechanism-labeled contaminated fields; the paired *permissive* view retains them. All train, validation, and test row assignments remain fixed between the two views.

3.1 Contamination Validity (C)

For field j, define

$$C_j = \mathbf{1}[j \text{ is invalid under the declared boundary at } t_p]\,. \tag{1}$$

C_j is fixed by task semantics and the mechanism contract. It is not inferred from mutual information, model importance, attribution, or downstream performance.

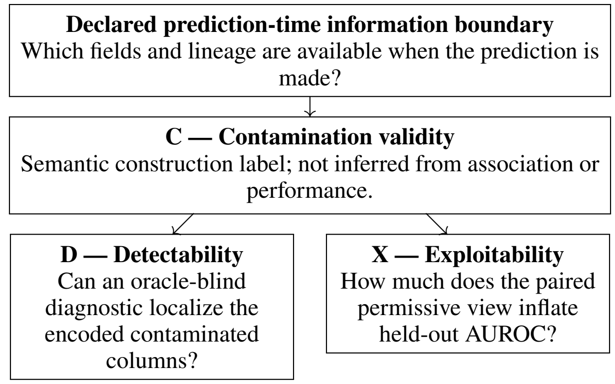

Figure 1: C is fixed by the declared information boundary. D and X are separate empirical measurements conditional on a diagnostic, representation, and learner; neither determines C or the other empirical axis.

3.2 Statistical Detectability (D)

An oracle-blind diagnostic assigns a score s_j to every encoded column without access to the contamination mask or test rows. Localization is evaluated against C_j with feature-level average precision (AP), normalized against contaminated-field prevalence π as

$$D = \frac{\mathrm{AP} - \pi}{1 - \pi}. \tag{2}$$

We also retain top-5 recall. Mutual information is the frozen primary ranker used in the headline C/D/X profiles and uses the same fixed `random_state=42` for every injected task. A sensitivity suite additionally uses absolute correlation, logistic coefficient magnitude, and random-forest permutation importance. MI, correlation, and logistic coefficients use training rows only; the forest is trained on training rows and scored by permutation on the frozen validation split. Each method's D is computed once per injected task and is therefore invariant to the downstream exploitability model. D is diagnostic- and representation-conditional; no per-task best-method maximum is a deployable selector.

3.3 Model Exploitability (X)

For learner m and a metric for which larger is better, paired harm is

$$X_m = q_m(\mathcal{D}_{\text{permissive}}) - q_m(\mathcal{D}_{\text{strict}}). \tag{3}$$

We use test AUROC as q. Positive X denotes measured performance inflation; near-zero X denotes no material benefit under this protocol. Negative X is not evidence that a learner recognizes or rejects invalidity.

4 LeakBench-Tab Design

4.1 Controlled Tasks and Mechanisms

The controlled registry contains 20 deterministic panel tasks spanning five data-generating archetypes. Each task includes ordered timestamps and sufficient entity and source support for the structured mechanisms. Generator seeds are separated from injection seeds, and every mechanism-strength-seed tuple has a content hash and a fixed chronological 60/20/20 split.

Table 1 groups mechanisms by construction rather than by observed performance. Strict-future mechanisms exclude the current row; M08 uses future entity outcomes with shrinkage and a leave-one-out prior; M09 uses outcome-dependent source collection with a complete one-hot representation and a random permutation of source category IDs to prevent ordinal encoding; and M10 keeps its legitimate component exactly equal to a clean input while labeling the contaminated component separately. These invariants make C auditable before any model is fitted. M09's D localizes eight encoded one-hot columns representing one source variable, so its detectability is representation-conditional rather than a field-level generality claim across encodings.

4.2 Models

The five core learners are logistic regression, random forest (Breiman 2001), CatBoost (Prokhorenkova et al. 2018), LightGBM (Ke et al. 2017), and the official TabM implementation (Gorishniy, Kotelnikov, and Babenko 2025). Scaling, preprocessing, and diagnostics are fit on training data only. The TabM tasks are serialized locally and verified by task, split, and bundle hashes before GPU fitting; no surrogate implementation or silent model fallback is accepted.

4.3 Real-Data Case Studies

Five fixed case studies—UCI Bank Marketing (UCI Machine Learning Repository 2014), a local Lending Club accepted-loan mirror, U.S. flight delays (U.S. Bureau of Transportation Statistics 2026), Chicago food inspections (City of Chicago 2026), and NYC 311 (City of New York 2026)—declare a prediction boundary and audit-confirmed post-boundary fields before evaluation. They use the four CPU core models. These studies fit categorical vocabularies on training rows only, map unseen validation/test levels to a reserved unknown value, and exclude globally encoded date strings. The earlier full-table-vocabulary run is superseded and cannot source claims. These studies ask whether paired harm appears under specific real task boundaries; they do not train a cross-task detector, test zero-shot metadata transfer, or justify population inference from five selected tasks. The Lending mirror is treated as local-only and non-redistributable until its source and license are verified.

5 Experimental Protocol

The frozen controlled matrix crosses 20 tasks, 11 mechanisms, 5 strengths, 5 learners, and 5 injection seeds, yielding 27,500 paired model-training cells. The primary estimand is mean paired AUROC harm. Clean-model results are cached only within an identical task/model/seed key, and every failed cell is retained rather than silently dropped.

The task is the independent unit. Confirmatory intervals use a 20,000-draw hierarchical bootstrap that resamples tasks and then injection seeds. Mechanism tests use task-level sign flips with Holm correction over the declared family; the three category contrasts use exact task-level sign flips and form a separate Holm family. M08 entity sensitivity shares each entity draw across all models and strengths within a task/seed group before outer resampling. M09 shares each source-category draw across models within a task/seed/strength group, but its eight source IDs are constructed categories rather than sampled population clusters; that interval is descriptive designed-category reweighting only. Because the 20 tasks are a designed registry rather than a random sample from a defined task population, these intervals quantify stability to registry reweighting; they do not support population inference to arbitrary tabular tasks. Association between mean D and X is explicitly descriptive: we report global rank correlation, category-only and category-plus-D fits, within-category permutation, and leave-one-mechanism-out (LOMO) diagnostics. The separate sensitivity matrix contains $20\times 11\times 5\times 5\times 4 = 22,000$ diagnostic cells on the same immutable tasks. It is complete only when all PENDING cells pass integrity checks; its rows are not added to the 27,500 model-training count.

Post-unblinding evidence reconciliation found that the original sensitivity runner had used the injection seed for its mutual-information rows, while the headline analysis and frozen task manifest used the fixed seed above. A scope-locked amendment therefore replaces all and only the 5,500 MI rows with the already-frozen task-manifest localization metrics; the other 16,500 rows and the method set are unchanged. The raw suite remains in the archive as provenance but is not paper-facing evidence.

A second post-unblinding methodological audit found that bootstrap tail areas had been labeled as category-contrast p-values, D–X intervals had resampled X while holding D fixed, and cluster draws had been made independently per model cell. A second audit then found that the initial cluster amendment still drew M08 entities independently across five injection seeds that reuse the same test rows, labels, and entities. The final scope-locked amendment replaces those summaries: exact task sign flips with Holm adjustment are used for category tests, D and X are jointly resampled, and cluster/category draws are synchronized at their shared task level. For M08, one entity draw is shared across all five seeds, models, and strengths within a dataset while seed-specific effects are retained for outer resampling. Prediction arrays and task hashes are checked directly against the frozen bundles. Mechanism-profile comparisons remain descriptive because no decision thresholds for them were bound by the original protocol; both superseded cluster summaries are barred from claim provenance.

Completion is not asserted in this draft because the final confirmatory claim release is absent.

6 Results

Blocked draft. No confirmatory result is rendered because `generated/result_macros.tex` has not been produced from a complete, confirmatory, non-pending `paper_claims.json`. Pilot estimates are deliberately excluded from the manuscript.

ID	Mechanism	Category	Invalidity source / structure
M01	Direct target copy	Simple	Target-derived global field
M02	Noisy target proxy	Simple	Noisy target-derived global field
M06	Redundant leakage cluster	Simple	Correlated target proxies
M10	Mixed leakage	Simple	Legitimate and contaminated components
M03	Nonlinear target interaction	Boundary	Nonlinear target-derived interaction
M07	Sparse subgroup leakage	Boundary	Contamination restricted to a subgroup
M11	Graph-mediated leakage	Boundary	Connected contamination component
M04	Post-outcome aggregation	Structured	Strictly post-outcome temporal aggregate
M05	Temporal look-ahead	Structured	Strictly future-window information
M08	Entity-conditioned leakage	Structured	Future entity-outcome statistic
M09	Source-conditioned leakage	Structured	Outcome-dependent source assignment

Table 1: Frozen mechanism taxonomy. Categories are declared by construction and are not reassigned after observing model performance.

7 Limitations and Conclusion

The controlled tasks are synthetic panel generators, not twenty real datasets. The registry covers eleven designed mechanisms and four oracle-blind, pre-test development rankers; neither set is exhaustive. AUROC harm addresses binary discrimination and does not replace calibration, decision-utility, or deployment monitoring. The five real-data audits are selected, boundary-specific case studies with sparse contamination labels, so their uncertainty intervals are descriptive and their conclusions do not generalize to an unspecified task population. Model coverage is limited to five core families, and differences between families are associational rather than causal. Metadata localization and governance comparisons remain outside this paper's confirmatory evidence.

The non-empirical conclusion is limited to the benchmark design; the final empirical conclusion remains blocked pending the confirmatory release.

References

Breiman, L. 2001. Random Forests. *Machine Learning*, 45: 5–32.

City of Chicago. 2026. Food Inspections. Chicago Data Portal. Accessed 2026-07-13.

City of New York. 2026. 311 Service Requests from 2020 to Present. NYC Open Data. Accessed 2026-07-13.

Davis, S. E.; Matheny, M. E.; Balu, S.; and Sendak, M. P. 2024. A Framework for Understanding Label Leakage in Machine Learning for Health Care. *Journal of the American Medical Informatics Association*, 31(1): 274–280.

Gardner, J.; Popovic, Z.; and Schmidt, L. 2023. Benchmarking Distribution Shift in Tabular Data with TableShift. In *Advances in Neural Information Processing Systems*, volume 36.

Gorishniy, Y.; Kotelnikov, A.; and Babenko, A. 2025. TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling. In *International Conference on Learning Representations*.

Grinsztajn, L.; Oyallon, E.; and Varoquaux, G. 2022. Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? In *Advances in Neural Information Processing Systems*, volume 35.

Kapoor, S.; and Narayanan, A. 2023. Leakage and the Reproducibility Crisis in Machine-Learning-Based Science. *Patterns*, 4(9): 100804.

Kaufman, S.; Rosset, S.; and Perlich, C. 2011. Leakage in Data Mining: Formulation, Detection, and Avoidance. In *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 556–563.

Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; and Liu, T.-Y. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, volume 30.

Magar, I.; and Schwartz, R. 2022. Data Contamination: From Memorization to Exploitation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 157–165.

McElfresh, D.; Khandagale, S.; Valverde, J.; C, V. P.; Ramakrishnan, G.; Goldblum, M.; and White, C. 2023. When Do Neural Nets Outperform Boosted Trees on Tabular Data? In *Advances in Neural Information Processing Systems*, volume 36.

Prokhorenkova, L.; Gusev, G.; Vorobev, A.; Dorogush, A. V.; and Gulin, A. 2018. CatBoost: Unbiased Boosting with Categorical Features. In *Advances in Neural Information Processing Systems*, volume 31.

Rosenblatt, M.; Tejavibulya, L.; Jiang, R.; Noble, S.; and Scheinost, D. 2024. Data Leakage Inflates Prediction Performance in Connectome-Based Machine Learning Models. *Nature Communications*, 15: 1829.

Roth, S. 2026. Which Leakage Types Matter? A Quantitative Landscape Across 2,047 Benchmark Datasets. arXiv:2604.04199.

Saito, T.; and Rehmsmeier, M. 2015. The Precision–Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 10(3): e0118432.

Subotić, P.; Bojanić, U.; and Stojić, M. 2022. Statically Detecting Data Leakages in Data Science Code. In *Proceedings of the 11th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*, 16–22.

UCI Machine Learning Repository. 2014. Bank Marketing. CC BY 4.0.

U.S. Bureau of Transportation Statistics. 2026. Reporting Carrier On-Time Performance Data. TranStats data portal. Accessed 2026-07-13.